FOOD CALORIE ESTIMATION FROM PHOTOGRAPHS

Complete Project Guide — Everything You Need to Know

Yashika (231031150057) | Pushpa (231031150040) | Atharv (231031150013)

Guide: Dr. Anu Yadav | B.Tech CSE (IoT) — 6th Semester

Gurugram University, Gurugram | Jan–May 2026

---

1. WHAT IS THIS PROJECT ABOUT?

This project is a web application that lets you take a photo of food and instantly tells you what the food is and how many calories it has. Think of it like a smart food diary — instead of manually typing "I ate a pizza", you just snap a photo and the app figures out everything automatically.

How does it work? (Simple version)

- You upload or take a photo of your food
- Our AI model (a trained neural network) looks at the image
- It figures out what food it is (e.g., "pizza", "sushi", "biryani")
- It then looks up the nutritional info from a database (USDA)
- It shows you: calories, protein, fat, carbs, and fiber

2. KEY TERMS EXPLAINED

(Google these too! Understanding these will help you in viva.)

Deep Learning

A type of artificial intelligence that uses "neural networks" — programs inspired by how the human brain works. Instead of writing rules like "if the image has red circles, it might be a tomato", deep learning **learns these patterns automatically** by looking at thousands of example images. The more examples it sees, the smarter it gets.

Convolutional Neural Network (CNN)

A special type of neural network designed specifically for images. It works by sliding small filters across the image to detect features — first simple things like edges and colors, then more complex things like textures, shapes, and finally whole objects (like a burger or a salad). Think of it like your eyes + brain working together.

EfficientNetV2-S

This is the specific CNN model we use. The "S" stands for "Small". It was created by Google researchers (Tan & Le, 2021). What makes it special is that it's very accurate but uses fewer computations than older models — like getting A+ grades while studying less than others. The "V2" means it's the second, improved version.

Transfer Learning

Instead of training a model from scratch (which needs millions of images and days of computing), we take a model that was **already trained on ImageNet** (14 million images of everyday objects). It already knows what edges, textures, and shapes look like. We just teach it the final step: "these shapes and textures = pizza" or "these = sushi". It's like hiring someone who already knows cooking and just teaching them your recipes, instead of teaching them what a knife is.

Food-101 Dataset

A famous dataset created by ETH Zurich (a top university in Switzerland). It contains **101,000 photos** of **101 different food types**. Each food type has 1,000 photos — 750 for training and 250 for testing. The foods range from "apple pie" to "waffles". It's the standard benchmark everyone uses to compare food recognition models.

ImageNet

A massive dataset of 14 million images across 1,000 categories (dogs, cars, buildings, etc). Our model was first trained on this, so it already understands general visual concepts before we show it food images.

Training (what the model does to learn)

Training means showing the model thousands of labeled images ("this is pizza", "this is sushi") and adjusting its internal parameters (called "weights") so it gets better at predicting. We train in **two phases**:

Phase 1 (Frozen Backbone): We freeze the existing knowledge and only train the new food-classification part. 5 epochs. Like learning the basics.

Phase 2 (Fine-Tuning): We unfreeze the top layers and let the whole model adjust. Up to 25 epochs. Like polishing your skills after learning the basics.

Epoch

One complete pass through ALL training images. If we have 75,750 training images and we do 5 epochs, the model sees each image 5 times. More epochs = more learning (up to a point — after that it starts "memorizing" instead of "understanding", which is called *overfitting*).

Accuracy — Top-1 vs Top-5

Top-1 accuracy: The model's #1 guess is correct. Our model gets ˜75%+ (great for 101 categories!).
Top-5 accuracy: The correct answer is somewhere in the model's top 5 guesses. Our model gets ˜93%+.

Why Top-5 matters: Sometimes "fried rice" and "biryani" look similar. If the model says "fried rice" as #1 but "biryani" as #2, Top-5 counts that as correct.

Loss Function (Cross-Entropy Loss)

A number that measures how wrong the model's predictions are. Lower = better. During training, the model tries to minimize this number. Think of it as an exam score, but inverted — 0 means perfect, higher means worse.

Learning Rate

How big of a step the model takes when adjusting its weights. Too big = it overshoots and never learns well. Too small = it learns too slowly. We use 0.001 for Phase 1 and 0.0001 for Phase 2. It's like adjusting the volume knob — big turns at first, then small fine adjustments.

Cosine Annealing

A strategy to gradually reduce the learning rate following a cosine curve. It starts at the set learning rate, slowly decreases, giving the model a chance to "settle in" to a good solution.

Early Stopping

If the model stops improving for 5 consecutive epochs (patience=5), we stop training early. This saves time and prevents overfitting.

Overfitting

When a model memorizes the training images instead of learning general patterns. Like a student who memorizes answers instead of understanding concepts — they ace practice tests but fail real exams. We prevent this with dropout, data augmentation, and early stopping.

Dropout (p=0.3)

During training, we randomly "turn off" 30% of neurons in the classifier. This forces the model to not rely on any single neuron too much, making it more robust. It's removed during actual predictions.

Data Augmentation

We artificially increase our training data by randomly flipping, rotating, cropping, and adjusting colors of images. So one pizza image becomes many slightly different pizza images. This helps the model generalize better.

USDA FoodData Central

The U.S. Department of Agriculture's database of nutritional information for thousands of foods. We use it to map each food class to its calorie count, protein, fat, carbs, and fiber values. It's the most trusted source for food nutrition data.

Flask

A lightweight Python web framework. We use it to create our web server — it receives the uploaded image, runs the AI model, and sends back the results. It's like the "backend brain" of our web app.

MPS (Metal Performance Shaders)

Apple's GPU acceleration framework. It lets us train our model on Mac GPUs (M1/M2/M3 chips) instead of needing an NVIDIA GPU. It's like CUDA but for Mac.

API Endpoint (/predict)

A URL on our web server that accepts requests. When you upload a photo, it gets sent to `localhost:5000/predict` the server processes it with the AI model, and returns the results as JSON data.

Backbone (in neural networks)

The main body of the neural network that extracts features from images. Think of it as the "eyes" of the model. The classifier head (the last layer) is the "brain" that makes the final decision.

3. HOW TO RUN THE PROJECT

Step 1: Install Python

Make sure you have Python 3.9 or higher. Check by typing in Terminal: `python3 --version`

Step 2: Install Dependencies

Open Terminal in the project folder and run: `pip3 install -r requirements.txt`
This installs PyTorch (AI framework), Flask (web server), Pillow (image processing), etc.

Step 3: Run the Web App

In Terminal: `python3 app.py`
Open your browser and go to: `http://localhost:5000`

Step 4: Use It

- Upload a food photo or use your camera
- Click "Analyze" and wait 2–3 seconds
- See the results: food name, confidence %, and nutrition info

4. PROJECT FILE STRUCTURE

File	What it does
`app.py`	Main web server. Loads trained model, handles image uploads, returns predictions.
`train.py`	Training script. Runs Phase 1 (frozen) and Phase 2 (fine-tuning) automatically.
`src/model.py`	Neural network architecture — EfficientNetV2-S with custom classifier head.
`src/dataset.py`	Loads Food-101 images and applies data augmentation.
`src/nutrition.py`	Nutritional data for all 101 food categories from USDA.
`templates/index.html`	Frontend UI — the webpage you see in the browser.
`models/best_model.pth`	The trained model file — the "brain" of our AI.
`requirements.txt`	Lists all Python packages needed.
`setup_data.py`	Script to extract the Food-101 dataset.

5. QUESTIONS YOUR TEACHER MIGHT ASK

Q: What is the main objective of your project?

A: To build a system that can identify food from a photograph and estimate its nutritional content (calories, protein, fat, carbs, fiber) using deep learning. The goal is to make dietary tracking automatic and effortless.

Q: Why did you choose EfficientNetV2 over other models like ResNet or VGG?

A: EfficientNetV2 achieves higher accuracy with fewer parameters and faster training. It uses compound scaling (balancing depth, width, and resolution) which is more efficient than just making networks deeper like ResNet or wider like VGG.

Q: What is transfer learning and why did you use it?

A: Transfer learning means using a model pre-trained on a large dataset (ImageNet, 14M images) and adapting it for our specific task. We used it because: 1) We don't have millions of food images, 2) It trains much faster, 3) It gives better accuracy than training from scratch. The model already knows general image features; we just teach it food-specific patterns.

Q: What dataset did you use? How big is it?

A: Food-101 dataset by ETH Zurich. 101,000 images total, 101 food categories, 1,000 images per category (750 training, 250 testing). It's the standard benchmark for food recognition research.

Q: What accuracy did your model achieve?

A: Top-1 accuracy: ˜75–80% (the model's best guess is correct). Top-5 accuracy: ˜93%+ (correct answer in top 5 predictions). This is competitive with published research on Food-101.

Q: Why two-phase training?

A: Phase 1 (frozen backbone, 5 epochs): Trains only the classifier head, preventing the randomly initialized classifier from corrupting pre-trained features. Phase 2 (fine-tuning, up to 25 epochs): Unfreezes top layers for food-specific feature adaptation, boosting accuracy from ˜54% to ˜75%+.

Q: Where does the nutritional data come from?

A: USDA FoodData Central — the official U.S. government database for food composition. We mapped each of the 101 food classes to its per-serving nutritional values.

Q: What technologies did you use?

A: Python for everything. PyTorch for deep learning. EfficientNetV2-S as the model. Flask for the web server. HTML/CSS/JavaScript for the frontend. Trained on Apple M1 Max using MPS (Metal Performance Shaders).

Q: What is overfitting and how did you prevent it?

A: Overfitting is when the model memorizes training images instead of learning general patterns. We prevented it with: 1) Dropout (30%), 2) Data augmentation (random flips, rotations, crops), 3) Early stopping (patience=5), 4) Transfer learning.

Q: Can it recognize Indian food?

A: The Food-101 dataset includes some Indian/Asian foods like samosa, fried rice, and edamame. For foods not in the 101 categories, it predicts the closest match. Future work could add Indian food datasets.

Q: What are the limitations?

A: 1) Limited to 101 food categories. 2) Cannot estimate portion sizes from images. 3) Struggles with mixed dishes. 4) Lighting/angle affect accuracy. 5) Nutritional values are averages.

Q: How does the web application work technically?

A: User uploads photo $\rightarrow$ Browser sends it to Flask server's /predict endpoint $\rightarrow$ Server preprocesses image (resize 224×224, normalize) $\rightarrow$ Feeds through EfficientNetV2-S $\rightarrow$ Gets probabilities for 101 classes $\rightarrow$ Picks top 3 $\rightarrow$ Looks up nutrition data $\rightarrow$ Returns JSON $\rightarrow$ Frontend displays results.

Q: What is cosine annealing?

A: A learning rate schedule that decreases the LR following a cosine curve from initial value to near zero. Allows large updates early (exploring solutions) and small updates later (fine-tuning). Often leads to better accuracy than a fixed LR.

Q: What is the difference between CUDA and MPS?

A: CUDA is NVIDIA's GPU computing platform (NVIDIA GPUs only). MPS is Apple's equivalent (Apple Silicon only). Both accelerate training via GPU. We used MPS on Apple M1 Max.

6. THINGS TO GOOGLE / YOUTUBE

- "What is a neural network" — 3Blue1Brown YouTube series (best visual explanation)
- "How CNNs work" — any animated YouTube explanation
- "EfficientNet explained" — understand compound scaling
- "Transfer learning explained simply"
- "Food-101 dataset" — original paper and examples
- "How Flask works Python" — understand the web server
- "What is an API endpoint" — how frontend talks to backend
- "Overfitting vs underfitting machine learning" — key viva concept
- "Data augmentation in deep learning" — why we flip and rotate images
- "Cross-entropy loss explained" — our loss function
- "Apple M1 MPS PyTorch" — how we train on Mac GPUs